

Catch Me if You Can: Effective Honeypot Placement in Dynamic AD Attack Graphs

Huy Q. Ngo
School of CMS
University of Adelaide
Adelaide, Australia
quanghuy.ngo@adelaide.edu.au

Mingyu Guo
School of CMS
University of Adelaide
Adelaide, Australia
mingyu.guo@adelaide.edu.au

Hung Nguyen
School of CMS
University of Adelaide
Adelaide, Australia
hung.nguyen@adelaide.edu.au

Abstract—We study a Stackelberg game between an attacker and a defender on large Active Directory (AD) attack graphs where the defender employs a set of honeypots to stop the attacker from reaching high-value targets. Contrary to existing works that focus on small and static attack graphs, AD graphs typically contain hundreds of thousands of nodes and edges and constantly change over time. We consider two types of attackers: a simple attacker who cannot observe honeypots and a competent attacker who can. To jointly solve the game, we propose a mixed-integer programming (MIP) formulation. We observed that the optimal blocking plan for static graphs performs poorly in dynamic graphs. To solve the dynamic graph problem, we re-design the mixed-integer programming formulation by combining m MIP (dyMIP(m)) instances to produce a near-optimal blocking plan. Furthermore, to handle a large number of dynamic graph instances, we use a clustering algorithm to efficiently find the m -most representative graph instances for a constant m (dyMIP(m)). We prove a lower bound on the optimal blocking strategy for dynamic graphs and show that our dyMIP(m) algorithms produce close to optimal results for a range of AD graphs under realistic conditions.

Index Terms—Active Directory, Attack Graph, Network Security, Honeypot Placement, Stackelberg Game

I. INTRODUCTION

Microsoft Active Directories (AD) are popular directory services for identity and access management and are deployed at most enterprises. Due to their popularity, AD systems have been major targets for attackers over the last decade. In 2021, Microsoft reported more than 25.6 billion brute force attacks on their AD accounts [3]. In these attacks, the attacker first builds an attack graph of the targeted AD system where nodes are user accounts, computers, security groups, etc. Each edge in the AD attack graph represents an existing exploit that the attacker can use to move from node to node. The attacker then uses the attack graph to escalate themselves from low privilege nodes to higher privilege nodes (ex. Account A $\xrightarrow{\text{AdminTo}}$ Computer B $\xrightarrow{\text{HasSession}}$ Account C) [5]. Tools for generating AD attack graphs are widely available, such as BloodHound¹. The same tools are also used by blue teams for defending AD systems. Defending AD systems, however, remains very challenging as AD systems are large, complex, and continuously evolve over time.

In this paper, we study a new method for defending AD by using active defense with honeypots. In our model, the defender places a set of honeypots on nodes of the AD attack graph. The attack campaign fails when the attacker goes into one of the honeypots. We assume that attackers use tools such as SharpHound² to build the AD attack graph and use the shortest paths from a given set of entry nodes to traverse to the Domain Admin (DA) node. Shortest-path attack plans are provided by default in BloodHound and are widely used by attackers in real attacks [1].

Active defense with honeypots is not new. However, the problem of placing honeypots in an AD attack graph represents two unique challenges that have not been studied thus far. The first challenge is the scale of the graphs. An AD attack graph typically consists of thousands of nodes and hundreds of thousands of edges with millions of attack paths, even for a small/medium organization. The second challenge is to develop a decoy solution that remains effective even when the graph dynamically changes. AD attack graphs are very dynamic. One of the major sources of changes in the AD graphs is users' activities such as logging on and off a workstation. In an AD attack graph, these dynamics are represented by a special type of edges called HasSession edges [4]. HasSession edges are added to the graph when a user signs on to a computer and has their credential stored in the computer memory. HasSession edges will stay online until being removed from the graph when the user signs off from the computer after a period of time.

Our solution is designed to be effective against attackers with different tactics and techniques. In our proposed model, we consider two types of attackers: a **simple attacker**, who is unable to detect any honeypots, and a **competent attacker**, who employs sophisticated detection tools to obtain complete visibility of all honeypots. In practice, it is difficult to ascertain the level of competence of an attacker. However, we can assume the probability of our network being attacked by one of these attackers based on the observation from previous attacks and/or analysis of the attacker's tactics using databases such as MITRE ATT&CK [2].

Our first proposal is to use a mixed-integer programming

¹<https://github.com/BloodHoundAD/BloodHound>

²<https://github.com/BloodHoundAD/SharpHound>

(MIP) method to solve the honeypot allocation problem for mixed-type attackers. For our problem, we observed that the Linear Programming approach can be very efficient in solving AD graphs due to the exploitability of MIP on the unique tree-like structure of AD graphs [8].

The above solution, however, assumes a static graph and performs poorly when the graph dynamically changes. Our approach to addressing the graph dynamic problem is to simulate all possible variations of the HasSession edges and design a placement solution that works best for all of these graph instances. This solution would only work for very small AD systems with a small number of HasSession edges as the number of graph instances grows exponentially with the number of these edges. Instead, we expand the MIP formulation to include m samples of the AD graph. We call it mixed-integer programming formulation for dynamic graphs (dyMIP(m)). As the run-time of dyMIP(m) grows for $m \gg 1$, we propose 2 heuristics (*voting-based heuristic* and *clustering-based heuristic*) that allow dyMIP(m) to approximate the optimal strategy very quickly in these cases. While *voting-based heuristic* is a very natural heuristic based on the idea of running smaller batches of samples on dyMIP(m), hence reducing the m value. *Clustering-based heuristic* approach hypothesizes that there is a number of good-quality samples in our graph sample that could be used to improve the solution for our dyMIP(m) algorithm. The novel idea of *clustering-based heuristic* is to solve combinatorial in each individual (static) graph to derive features and then apply a clustering algorithm to improve the quality of the sample for the general problem on a dynamic graph.

Our key contributions are

- We formulate a general honeypot placement problem in AD graphs considering 2 types of attackers. We show that the static graph version of our problem is already NP-hard when attackers are unable to detect any honeypots and $W[1]$ -hard when attackers have complete visibility of all honeypots.
- We provide a mixed integer program formulation and solutions for placing honeypots in very large AD graphs. Our solution could solve the problem on graphs of sizes that none of the previous approaches could;
- We study for the first time the honeypot placement problem with dynamic graphs. All of the existing approaches focused only on static graphs. We develop two heuristics based on voting and clustering to solve the dynamic graph problems. Our solutions, especially the clustering-based method, which derives defense based on “representative” snapshots of the dynamic graph (cluster centroids), run very fast on large graphs and provide close to optimal results. Our experiment will consider the algorithm on a range of AD graphs under realistic conditions.

II. BACKGROUND AND RELATED WORK

Guo et al. [8] study the shortest path edge interdiction problem on AD attack graphs. The authors propose a dynamic programming-based tree decomposition method to derive an

optimal solution for their edge-blocking problem in directed acyclic AD graphs. They then developed a Graph Neural Network as a heuristic to enhance the scalability of their solution. Guo et al. [9] extend their previous edge-blocking problem by proposing several scalable algorithms including Reinforcement Learning for tree decomposition and mixed-integer programming (MIP) algorithms. The authors later extend the tree decomposition algorithm to a general AD graph with cycles and derive the mixed strategy solution for their problem via MIP. In addition, Zhang et al. [10] showed the scalability of a double oracle-based algorithm on the same problem. Goel et al. [6] assumed the detection and failure probability of attackers on every edge of the AD graph. They deploy a neural network to approximate the attacker’s strategy and apply an evolutionary diversity optimization (EDO) strategy to solve the defender problem. In [7], Goel et al. further improved their defense algorithm (EDO) by improving attacker strategy (fitness function) via Reinforcement Learning. These works, however, only work on static graphs and are not applicable to real AD networks, which are naturally dynamic. One of our key observations in this paper is that optimal strategies on a static graph perform poorly when applied to dynamic networks. Our main difference with these existing works on AD graphs is that our algorithms are designed for dynamic graphs and hence generate better defenses for real AD systems.

Honeypots have been widely studied in the literature. Stackelberg games are typically used to formulate the honeypot allocation problem in previous works [11]–[14]. The models in these studies are very similar to ours: they use attack graphs to model the attacker’s actions and the defender can stop the attacker by deploying honeypots in the network. The defender’s task is to minimize the damage caused by the attacker with the minimum cost of honeypot deployment. In [14], Milani et al. study games where the defender can either deploy honey-edge or remove an edge from a network to lure the attacker to honeypots. Although these studies can optimally place honeypots in the network, they only work on very small graphs and suffer scalability issues. Indeed, in all of these works, they only consider networks of a hundred nodes in their experiments. Real AD graphs are several orders of magnitude larger than these graphs. Our solutions in this paper are the first solutions that can effectively handle honeypot placement at that scale.

More recently, Shinde et al. [16] model the cyber deception game as an interactive partially observable Markov decision process. The authors of [16] focus on luring the attacker to fake data by allocating deception files on network hosts - that is when the attacker already gets in the honeypot. Our work focuses on finding the optimal honeypot allocation strategy at the network level. In [15], Lukas et al. study the honeypot placement in AD graphs. The author proposed to use deep learning variational autoencoder model to generate and place honey users in the network. Their focus is on the topological structure of the fake AD graphs, not the impact of their solution on the attacker’s success rate - a real measure of effectiveness for these solutions. Again, these recent solutions

only consider honeypot allocation in small and static networks.

In this paper, we aim to provide practical solutions for allocating honeypots on large and dynamic AD attack graphs against realistic attackers.

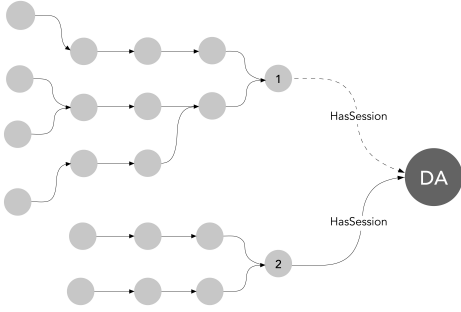


Fig. 1: An attack graph with dynamic HasSession edges.

Fig. 1 shows an attack graph of a typical dynamic AD environment. There are two HasSession edges to the DA (The DA uses the edge $1 \rightarrow \text{DA}$ regularly but only logs into 2 when needed). A static snapshot of the graph typically captures only the branch that has $1 \rightarrow \text{DA}$ and completely misses the branch of the graph that contains $2 \rightarrow \text{DA}$. An attacker would wait in a system for a long time to wait for edge $2 \rightarrow \text{DA}$ to appear and can bypass static defenses built on only the $1 \rightarrow \text{DA}$ branch. None of the previous works on AD attack graphs [5]–[10] considered dynamic AD graphs and would fail to defend against this type of attacks.

Our solution can provide near-optimal honeypot allocation in dynamic networks with hundreds of thousands of nodes and millions of edges.

III. PROBLEM AND MODEL DESCRIPTION

A. Honeypot placement model for static graphs

We consider a Stackelberg game between an attacker and a defender on a directed AD attack graph $G = (V, E)$. There is a set $S \subseteq V$ of entry nodes, and the attacker can enter the graph via one of $s = |S|$ entry nodes. The attacker's ultimate goal is to reach the Domain Admin (DA). The attacker performs attacks by choosing paths with pure actions. (i.e., an attacker can run BloodHound [4] to generate the paths to DA). From an entry node, there are many possible attack paths to DA, and the attacker can choose to attack the DA. Also, there is a fixed set of *blockable* nodes, denoted by $N_b \subseteq V$, the defender's task is to pick b nodes in N_b to allocate honeypots in order to intercept as many of the attacker's *attack paths* as possible. A honeypot will stop the attacker from going further into the network, that is, if the attacker stumbles into a honeypot, the attack campaign fails. We study two types of attacker models, the **simple attacker** and **competent attacker**. We introduce a defending problem denoted as $HATK(\varphi)$, where φ ($0 \leq \varphi \leq 1$) and represents the fraction of competent attacker agents ($1 - \varphi$ is the probability of simple attacker agents). When $\varphi = 1$, we focus on defending against pure

competent attackers, whereas $\varphi = 0$ represents defending against pure simple attackers.

In the simple attacker model, the attacker is assumed to be unable to differentiate between a normal node and a honeypot. The simple attacker will perform the shortest attack path. Therefore, the attacker's optimal strategy is to randomly select one of the shortest paths to the DA. The defender's goal is to reduce the number of "clean paths", which are paths that can be used by the attacker to reach the target without any intervention. The defender achieves this by applying a blocking plan B that reduces the number of clean paths. Let's denote by y_i the total number of clean paths from entry node i to DA and y_i^B as the remaining clean paths after the defender applies a blocking plan B . The attacker's success probability is $\frac{y_i^B}{y_i}$, and the expected success probability can be obtained by averaging over all entry nodes. Our optimization problem for allocating honeypot against the simple attacker is formally

defined as:

$$\min_{B \subseteq N_b, |B| \leq b} \sum_{i=1}^s \frac{y_i^B}{y_i}$$

In the competent attacker model, the attacker is capable of detecting every honeypot in the attack graph. Thus, when the attacker encounters a honeypot on its way to the target destination, it alternates to other paths. To prevent a competent attacker from reaching the DA, the defender must place honeypots to completely disconnect an entry node from the DA. The defender's goal is to maximize the number of entry nodes that are disconnected from the DA using a limited budget. Let's denote R_i as the reachability of node i to DA and given that allocating honeypot can change the reachability, we define R_i^B as the reachability of node i after applying blocking plan B . Our optimization problem for allocating honeypot against the competent attacker is formally defined as: $\min_{B \subseteq N_b, |B| \leq b} \sum_{i=1}^s R_i^B$. Combining both of the attacker model, the problem $HATK(\varphi)$ can be interpreted as:

$$\min_{B \subseteq N_b, |B| \leq b} \sum_{i=1}^s (\varphi R_i^B + (1 - \varphi) \frac{y_i^B}{y_i}) \quad (1)$$

To simplify notation, we introduce the following denotation: $z_{\varphi,i} = \varphi R_i^B + (1 - \varphi) \frac{y_i^B}{y_i}$, where $z_{\varphi,i}$ represents the normalized attacker success rate of reaching the Domain Admin (DA) from node i given a specific φ value.

The two theorems below establish the computational hardness of the honeypot placement problems presented in this paper. Due to space constraints and anonymity, we omit the proofs of these two theorems in this submission. Detailed proofs will be provided in the extended technical report, supplementing the main manuscript.

Theorem 1. Let L be the maximum shortest path length from any entry node to DA. The static version of the optimal honeypot placement problem against **Simple Attacker** is NP-hard when $L \geq 7$.

Theorem 2. The static version of the optimal honeypot placement problem against **Competent Attacker** is $W[1]$ -hard with respect to budget b .

B. Honeygot placement model for dynamic graphs

The above formulation only optimizes the honeygot placement for a given graph instance G . In real AD networks, the graph G changes constantly due to users' activities. In this paper, we consider dynamic graphs with on/off HasSession edges - a major source of dynamism in AD graphs. We model the dynamic graph process as follows.

There is a subset of edges $H \subseteq E$ (the HasSession edges) where each edge is turned on and off randomly. Formally, let $Comp$ be the set of computers and $User$ be the set of users in the network. The authentication data is represented in the tuple format $\langle t_s, t_e, u_i, c_j \rangle$, where t_s is the sign-in (start time), t_e is the sign-off time (end time), and u_i and c_j are a user and a computer in $User$ and $Comp$, respectively. This tuple indicates that user u_i signs in to computer c_j at time t_s and signs off that computer at time t_e . The log-on and log-off time of a session are readily available in typical Windows logs and can also be captured using tools such as SharpHound by attackers. A Hassession between a computer c_j and a user u_i appears in the attack graph g_t at time t ($(c_j, u_i) \in E_t$) if $t_s \leq t \leq t_e$.

At time t , a snapshot/realization of the dynamic graph is denoted by $g_t = (V, E_t)$, where $E_t = \langle e_{0,t}, e_{1,t}, \dots, e_{m,t} \rangle$ with $e_{i,t}$ referring to the state of a unique Hassession edge e_1 at time t . Specifically, $e_{i,t} = 1$ if the edge e_i is online at time t , otherwise $e_{i,t} = 0$.

Note that only the set of edges E_t changes in g_t . The set of nodes V remains the same. We denote the set of all graph instances as $G_s = \{g_1, g_2, g_3, \dots, g_m\}$, where m is the number of possible instances. In the dynamic setting, given a defensive budget of b , the defender's problem is to allocate honeypots to limit the attacker's clean paths on every possible snapshot/realization of the attack graphs. The dynamic version of the $HATK(\varphi)$ problem can be interpreted:

$$\min_{B \subseteq N_b, |B| \leq b} \sum_{g \in G_s} \sum_{i=1}^s (\varphi R_{i,g}^B + (1 - \varphi) \frac{y_{i,g}^B}{y_{i,g}}) \quad (2)$$

Again, to simplify the notation, we have: $z_{\varphi,i,g} = \varphi R_{i,g}^B + (1 - \varphi) \frac{y_{i,g}^B}{y_{i,g}}$. Note that we do not make any assumption on the temporal pattern of the edge sets in our solutions. We will show in the evaluation section that our algorithms are robust to dynamic graphs with realistic patterns.

IV. HONEYPOT PLACEMENT SOLUTIONS

A. Solution for static AD graphs

1) *Mixed-integer programming for static AD graphs:*
Firstly, we formulate a nonlinear program for solving the $HATK(\varphi)$:

$$\min \sum_{i=1}^s z_{\varphi,i}$$

$$y_i^B = \sum_{j \in n(i)} y_j^B, \quad \forall i \in V \setminus N_b \quad (3a)$$

$$y_i^B = \sum_{j \in n(i)} (1 - B_i) y_j^B, \quad \forall i \in N_b \quad (3b)$$

$$y_i^B \leq y_i, \quad \forall i \in V \quad (3c)$$

$$y_i^B \geq 0, \quad \forall i \in V \quad (3d)$$

$$R_i^B \geq R_j^B - B_i, \quad (i, j) \in E, i \in N_b \quad (3e)$$

$$R_i^B \geq R_j^B, \quad (i, j) \in E, \quad i \in N \setminus N_b \quad (3f)$$

$$\sum_{i \in V} B_i \leq b \quad (3g)$$

$$B_i, R_i \in \{0, 1\} \quad (3h)$$

$$R_{DA}, y_{DA} = 1 \quad (3i)$$

Where $n(i)$ is the set of the successor node of node i . The design of the formulation is based on the problem of defending against each individual attacker. We formulate it as a nonlinear program that is tailored to each type of agent. In our formulation, we use B_i to denote the unit of budget spent on node i , B_i is binary. Formally, $B_i = 0$ if $i \notin N_b$ and $B_i \in \{0, 1\}$ if $i \in N_b$.

Constraints (3a) to (3d) present the problem of defending against the simple attacker. The design of this set of constraints concentrates on computing the number of paths from an arbitrary node to the target (DA) on an **all-shortest path** graph by summing the number of paths to DA from each of its successors. We illustrate this idea using the example in Fig. 2. To calculate the number of paths to DA from 5 or y_5 , we need to recursively calculate y_3 , y_2 , y_1 and y_{DA} . First, we define that $y_{DA} = 1$ (there is 1 path from DA to itself). Then each node 1 and 2 has exactly 1 direct edge to DA so $y_1 = 1$ and $y_2 = 1$. Next, node 3 has two outgoing edges to nodes 2 and 1, meaning that attackers have two possible ways to reach DA via these nodes. Therefore, $y_3 = y_1 + y_2 = 1 + 1 = 2$. Finally, since node 5 has one outgoing edge to node 3, which could lead to two possible paths to DA, we set $y_5 = y_3 = 2$. Note that our path-calculating approach will fail in the presence of cycles in the graph, as the number of paths of a node in a cycle can go to infinity. Since the simple attacker is unable to observe any honeygot and they always go for the shortest paths, it is without loss of generality to consider only **all-shortest path** graphs, which is naturally acyclic.

In the case of defending against a competent attacker, we aim to reduce the attacker's success rate by disconnecting the maximum number of entry nodes from DA. Constraints (3e) and (3f) presented the problem of defending against a competent attacker. These constraints solve the problem based on the idea that if any of the successor nodes of node i can

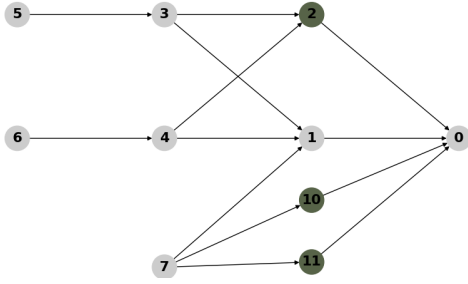


Fig. 2: An example all-shortest path AD attack graph. Nodes 5, 6, 7 are entry nodes. Nodes 0 is the DA. **Bold** nodes 2, 10, 11 are not blockable nodes

reach DA, then DA is also reachable via node i . We assume that DA is reachable by all entry nodes by default.

Our formulation is nonlinear due to the nonlinear constraint (3b). To obtain a linear version, we follow several transformation steps. First, we will replace y_i by an auxiliary variable v_i by adding constraints $y_i^B \leq v_i$, (3a) also become $v_i = \sum_{j \in n(i)} y_j^B$. Next, we replace (3b) by 2 additional constraints, $y_i^B \leq y_i \cdot (1 - B_i)$ and $y_i^B \geq v_i + y_i \cdot (-B_i)$. These constraints imply that if node i is blocked, then $y_i^B = 0$; otherwise, we compute y_i^B normally ($y_i^B = v_i = \sum_{j \in n(i)} y_j^B$).

B. Honey-pot Placement for dynamic networks

1) *Multi-graph Mixed-integer Programming*: For dynamic graphs, we require a different approach as we need to design a blocking plan that performs well against an exponential number of possible graph instances. Assume that we have a collection of “sample static instances” (snapshots) based on the dynamic AD graph, denoted by G_s . As shown in section III, we model the node blocking problem as a global optimization problem where the defender tries to minimize the attacker’s average success probability over the sample instances in G_s . Let us denote by m the number of graph samples in G_s . A naive approach is to merge all the graphs in G_s (i.e., find a super graph that contains all instances, essentially assuming all HasSession edges are on) and run sMIP on it. Of course, the resulting blocking plan will not be optimal.

Let $OBJ(g_i)$ be the objective function on graph $g_i \in G_s$ or the attacker’s success probability facing graph $g_i \in G_s$ in the static scenario. We also define $OBJ_m(G_s)$ to be the sum of m objective functions or the sum of the attacker’s success probabilities for all sample graphs in G_s (there are $m = |G_s|$ samples). Optimizing blocking strategy for dynamic graphs then boils down to minimizing the following objective function:

$$OBJ_m(G_s) = OBJ(g_1) + OBJ(g_2) + \dots + OBJ(g_m) \quad (4)$$

In practice, some snapshots of the graph will have a higher chance of appearing, in that case, we could adjust our objective function by multiplying by a weight k to that graph’s objective function (i.e., $k * OBJ(g_i)$). We have the following nonlinear program for the dynamic version:

$$\min \sum_{g \in G_s} \sum_{i=1}^s z_{\varphi, i, g}$$

$$y_{i,g}^B = \sum_{j \in n(i)} y_{j,g}^B, \quad \forall i \in V_g \setminus N_b, g \in G_s \quad (5a)$$

$$y_{i,g}^B = \sum_{j \in n(i)} (1 - B_i) y_{j,g}^B, \quad \forall i \in N_b, g \in G_s \quad (5b)$$

$$y_{i,g}^B \leq y_{i,g}, \quad \forall i \in V_g, g \in G_s \quad (5c)$$

$$y_{i,g}^B \geq 0, \quad \forall i \in V_g, g \in G_s \quad (5d)$$

$$R_{i,g}^B \geq R_{j,g}^B - B_i, \quad \forall (i, j) \in E_g, \quad i \in N_b, g \in G_s \quad (5e)$$

$$R_{i,g}^B \geq R_{j,g}^B, \quad \forall (i, j) \in E_g, g \in G_s, \quad i \in N \setminus N_b \quad (5f)$$

$$\sum_{i \in V} B_i \leq b \quad (5g)$$

$$B_i, R_{i,g} \in \{0, 1\} \quad (5h)$$

$$R_{DA}, y_{DA} = 1 \quad (5i)$$

The above formulation can be converted to a linear program, similar to the approach involving static graphs. In the rest of this paper, we will refer to the Mixed Integer Programming algorithm for dynamic graphs as $MIP_m(X)$, where X represents the set of graph instances considered in MIP and m denotes the cardinality of X . We use the term $dyMIP(m)$ when we do not need to specify the set of graphs under consideration.

2) *Optimal Lower Bound*: In theory, we can find the optimal blocking policy by running $dyMIP(m)$ algorithm on every possible graph instance. However, the linear program $MIP_m(X)$ requires $\mathcal{O}(m \cdot (n + e))$ variables, where n is the number of nodes and e is the number of edges in the graph $G \in X$. As a result, $MIP_m(X)$ becomes increasingly computationally challenging as m grows larger. To overcome this issue, we split G_s into j equally-sized batches, each batch has t graphs (i.e., $t \cdot j = |G_s|$). We denote each batch of graphs by $X_i \subset G_s$, and $X = \{X_1, X_2, \dots, X_j\}$ is the (all disjoint) batch set, where $G_s = X_1 \cup X_2 \cup \dots \cup X_j$. The following proposition shows the derivation of a Lower Bound on the overall objective using this batching approach.

Proposition 1. The batching solution gives a lower bound on the overall objective:

$$\sum_{X_i \in X} \min_B OBJ_t(X_i) \leq \min_B OBJ_m(G_s) = OPT \quad (6)$$

Again, we omit the proof for space constraints.

For (6), the left-hand side consists of sum of the attacker’s success probabilities for all batches, and the defender can apply different optimal blocking plans for different batches. The right-hand side is the sum of the attacker’s success probabilities for all samples, under the defender’s optimal blocking plan that is applied to all samples (i.e., the same

plan shared over all batches). The inequality in (6) provides a lower bound on the optimal result for the set of samples G_s . This lower bound can be estimated by summing the optimal results of each batch in X . Even for large values of m , this approach allows us to find the estimated lower bound with minimal computational effort. In simpler terms, solving a single instance of $MIP_{1000}(G_s)$ is more challenging than solving 100 instances of $MIP_{10}(X_i)$. By summing the objective values of all instances of $MIP_{10}(X_i)$, we are guaranteed to get a lower bound for the original problem. Furthermore, we could interpret the result of each batch as *an independent draw from the same (unknown) distribution* and our task is to calculate the mean based on sample averages, so using Hoeffding's Inequality Bound (more details presented below), we can achieve a high-confidence estimation without going over all batches.

3) *Generalization to Large Collections of Graph Samples*: As mentioned above, dyMIP(m) only works on a small number of graphs before running into scalability obstacles. We need to develop a better strategy for scaling dyMIP(m) to large m . We develop two heuristics for this purpose.

Voting based heuristic

The overall idea behind our *voting based heuristic* is to make use of the lower bound calculation in Section IV-B1. First, we divide G_s into j equal disjoint batches and run $MIP_t(X_i)$ on each batch and run each $MIP_t(X_i)$ to derive a blocking strategy B_i . Denote by $B = \{B_1, B_2, \dots, B_j\}$ (the set of blocking strategies generated based on all $MIP_t(X_i)$ instances). We use $C = (c(1), c(2), \dots, c(n))$ to count the number of times an individual node n is blocked (selected as a honeypot) under the blocking plans in B . We then sort C in descending order and pick the top b nodes in C as the final blocking/honeypot placement strategy. In other words, we solve for j separate blocking plans based on j batches, and use "majority voting" to come up with a blocking plan (find the b most voted nodes to place honeypots).

Clustering based heuristic

Our second heuristic is based on the idea that there is a set of most *representative* graph instances (snapshots, realizations) for the whole collection of samples. This approach is particularly useful when there are duplicate graphs in a set of samples or samples with similar structures. Our idea is only to run dyMIP(m) once on this *representative* set of graphs. To do that, we apply a clustering algorithm to explore different structures among the graph samples and group graphs with common structures into the same group. Formally, Let's define $feature(g)$ as features of graph g : $feature(g) = (f(1, g), f(2, g), \dots, f(s, g))$, where $f(i, g) = \frac{y_{i,g}^{B_g}}{y_{i,g}}$ represents the attacker's success rate when they are at a starting node i and facing a node blocking strategy B_g . B_g is produced by applying staticMIP on graph g . We use k-means as the clustering algorithm in our implementation. After getting $feature(g)$ for every graph $g \in G_s$, we apply

k-means to cluster G_s into k clusters. We consider graphs that are closest to centroids as the representative graphs for our dyMIP(m), where the distances to centroid are Euclidean distances [17]. We apply dyMIP(m) on the set of representative graphs that we got from the clustering algorithm.

There is a complication with the clustering algorithm where clusters are of varying sizes and densities. If we uniformly pick a graph from each cluster, then the algorithm is prone to be biased to the "fringe" clusters which degrade the quality of our samples. Our solution is to weigh the graph selection based on the cluster size. That is, when choosing a graph for dyMIP(m), we first choose m clusters where each cluster c with size $size_c$ has a probability $\frac{size_c}{|G_s|}$ for being picked. We then pick a graph instance that has the closest distance to centroid c to be the *representative graph*. The already chosen graph instances are removed from consideration to avoid duplication. *Clustering heuristic* is more scalable than *voting heuristic*. For example, suppose our batch size is 100, while *voting heuristic* requires us to run dyMIP(100) multiple times (majority voting does not make sense if there is only one vote), the *clustering heuristic* only requires us to run MIP(100) once.

V. EVALUATION

We evaluate our proposed solutions on both synthetic AD graphs with both randomized HasSession edges and real HasSession data. All of the experiments are carried out on a high-performance computing cluster with 2 CPUs and 24GB of RAM allocated to each task for the experiments. Code and data will be made publicly available.

A. Synthetic Graph Generation

We start with synthetic graphs generated by DBCreator³ and Adsimulator⁴ - two state of the art tools for creating AD graphs. In particular, we present results for the 5 synthetic graphs in Table I). The graphs labeled as "Rxxxx" were generated using DBCreator, while the ones labeled as "ADxxx" were created using Adsimulator. DBCreator allows us to generate graphs given a number of computers in the targetted AD network (R2000 and R4000 have 2000 and 4000 computer nodes, respectively). Adsimulator provides more flexibility by allowing graphs with specific properties, including the number of users, computers, domain trusts, OUs, GPOs, and more. ADX05 and ADX10 are created by increasing all its default parameters by a factor of 5 and 10 respectively. ADU10, ADU100 are created by tuning the parameters to mimic the real AD network of *University of Anonymized* which are 10%, 100% of the size of the mimicked network, respectively.

We perform the following pre-processing steps, First, when there are multiple DAs in the graph, we simply merge it into 1 unique DA. We only consider 3 types of edges in our graph: AdminTo, HasSession, and MemberOf. We only consider an entry node that can reach DA. ADU100 is the largest graph in

³<https://github.com/BloodHoundAD/BloodHound-Tools/tree/master/DBCreator>

⁴<https://github.com/nicolas-carolo/adsimulator>

our experiment with 137207 nodes and 1410679 edges. Table ?? show the size of each graphs

B. Experiments on static graphs

1) *Experiment Set Up:* We will refer mixed-integer programming approach as sMIP (or staticMIP). We also adopt 2 algorithms referred to as sMIP-S, sMIP-C where the former is sMIP formulation for solving the problem of (pure) simple attacker agent ($\varphi = 0$), and the latter is sMIP for solving the problem of (pure) competent attacker agent ($\varphi = 1$). Throughout our experiment, we assume $\varphi = 0.5$ (equal chance of being attacked by either of the attackers). However, it is important to note that our technique remains effective for any value of φ .

We use Gurobi 9.0.2 solver for solving staticMIP. For each graph, we run 10 trials and report the results for these runs. For each trial, we randomly draw 50 entry nodes and set the budget b to 10. We assumed only computer nodes are blockable (i.e., honeypots can be placed only on computers). In the experiment, we introduce 3 metrics to evaluate our algorithms: **Simple-Agent Success Rate (SSR)** and **Competent-Agent Success Rate (CSR)** and **Mean Success Rate (MSR)**. SSR and CSR are the success rates of the Simple and Competent attacker agent when facing a blocking strategy from the defender. SSR is measured as the fraction of the clean paths over the number of paths in the all-shortest path graph, normalized by the number of starting nodes and CSR is the fraction of entry nodes that can reach DA over the total number of entry nodes; whereas MSR is the average of SSR and CSR.

We carry out 2 sets of experiments. The first experiment which shows in Table I compares our sMIP-S and sMIP-C with 2 greedy algorithms namely GREEDY-S, GREEDY-C. Additionally, we evaluated our approach against Zhang et al.'s Double Oracle [10] (ZDO) algorithm. ZDO is the state-of-the-art edge-blocking algorithm for AD graphs. We adopt this approach for honeypot placement by formulating it as a simple blocking problem. The purpose of this comparison is to demonstrate that the existing techniques are not suitable for our honeypot placement problem without compromising its performance.

In the GREEDY-S algorithm, for each node, we can enumerate the shortest path that goes through it or calculate the betweenness centrality for each node. The idea is to iteratively block the node that has the highest betweenness centrality and repeat it b times. GREEDY-C uses a minimum node cut algorithm to disconnect each entry node and then prioritizes the entry nodes requiring the least budget to be blocked until the budget is depleted. In the second experiment shown in Table II, we simply show the performance of our sMIP against a scenario of 2 types of attacker agents in the network.

2) *Result Interpretation:* In Table I, it can be seen that sMIP-S and sMIP-C perform at least as well as, and often better than, GREEDYs on various graphs. While ZDO performed badly since it is designed for the shortest path interdiction problem. In Table II, our goal is to produce a defence plan that can be effective against both types of attacker agents. As they

TABLE I: Comparing sMIP algorithm against GREEDY algorithms. SSR is for simple attackers and CSR metric is used for competent attackers. Here $b = 10$.

	Simple Attacker (SSR)			Competent Attacker (CSR)	
	sMIP-S	GREEDY-S	ZDO	sMIP-C	GREEDY-C
R2000	0.233 (0.15s)	0.233 (0.01s)	0.389 (0.67s)	0.354 (0.09s)	0.542 (0.37s)
R4000	0.279 (0.10s)	0.279 (0.01s)	1.000 (0.13s)	0.530 (0.28s)	0.560 (0.48s)
ADX05	0.135 (0.12s)	0.136 (0.02s)	0.272 (0.68s)	0.670 (0.12s)	0.682 (1.04s)
ADX10	0.144 (0.16s)	0.155 (0.03s)	0.226 (0.48s)	0.606 (0.20s)	0.774 (1.86s)
ADU10	0.547 (1.27s)	0.547 (0.27s)	0.717 (0.54s)	0.984 (1.29s)	0.984 (24.5s)
ADU100	0.602 (15.3s)	0.602 (3.80s)	0.760 (0.15s)	1 (-)	1 (-)

TABLE II: Comparison between variants of sMIP algorithm. Optimal results are in **bold**.

	sMIP-S			sMIP-M			sMIP-C		
	SSR	CSR	MSR	SSR	CSR	MSR	SSR	CSR	MSR
R2000	0.233	0.682	0.46	0.303	0.356	0.330	0.313	0.354	0.334
R4000	0.279	0.856	0.568	0.421	0.536	0.478	0.452	0.53	0.498
ADX05	0.135	0.958	0.547	0.17	0.794	0.482	0.492	0.67	0.581
ADX10	0.144	0.992	0.568	0.409	0.63	0.519	0.468	0.606	0.539
ADU10	0.547	1	0.774	0.547	1	0.774	0.912	0.984	0.953
ADU100	0.602	1	0.801	0.602	1	0.801	0.851	1	0.925

do not make any relaxation in the MIP formulation, both sMIP-S and sMIP-C obtained the optimal defense against a Simple attacker and a Competent attacker, respectively. As shown sMIP-M significantly improves the MSR for $\varphi = 0.5$. This is not surprising as it is specifically designed to defend against both types of attackers. In *ADU100*, 10 honeypots are not enough to stop the competent attacker from any entry nodes hence, the attacker success rate for the competent attacker is 1. More significantly, staticMIP runs fast enough for large graphs and could scale to graphs of sizes equivalent to our anonymous network. That is, staticMIP is able to produce optimal results for very large static graphs, making it scalable for realistic AD graphs. Note that even though the general problem is NP-hard, what we observe here is that AD graphs have certain tree-like structures [8] that allow staticMIP to solve them very fast. In fact, the defending problem against both types of attackers is polynomially solvable if the AD graph is a tree (via a top-down Dynamic Program from DA to allocate honeypots). This makes the Linear Programming approach very efficient against problems on tree-like AD graphs.

C. Experiments on synthetic dynamic graphs

In Table III, we present our experiments on the *ADX05* graph with a budget of 20. Only computer nodes are blockable. We consider all user nodes in the graph as the entry nodes. After preprocessing, the numbers of user nodes and computer nodes are 72 and 500 respectively. Each edge is randomly ON in each run with probability $p = 0.5$. Ideally, we will run our dyMIP(m) on every possible instance to get the optimal blocking policy. However, this is computationally infeasible. A more practical way is to derive a blocking policy on a training set (sampled from the network) and evaluate it on a different testing set (future states of the network). In our experiments, we use 5,000 instances for training and run a Monte Carlo simulation on 100,000 graphs to evaluate the effectiveness of our blocking plan to ensure statistically significant results for the evaluation.

TABLE III: Experiments on dynamic graphs ADS05. We show the success rate of each algorithm with respect to their intended attacker (e.x. We show the Simple attacker Success Rate (SSR) for dyMIP-S algorithm, etc.)

		$m = 1^a$	$m = 10$	$m = 50$	$m = 100$
dyMIP-S	Random	0.7960 (2s)	0.7430 (16s)	0.7359 (89s)	0.7326 (174s)
	Kmean	0.7304 (10232s)	0.7409 (919s)	0.7343 (987s)	0.7325 (1062s)
dyMIP-M	Random	0.8851 (2s)	0.8494 (17s)	0.8461 (98s)	0.8407 (216s)
	Kmean	0.8417 (2416)	0.8544 (741s)	0.8398 (1011s)	0.8373 (1310s)
dyMIP-C	Random	0.9722 (2s)	0.9028 (16s)	0.8889 (95s)	0.8889 (188s)
	Kmean	0.9042 (9188s)	0.9000 (949s)	0.8889 (1010s)	0.8889 (1069s)
	Vote		0.9028 (8718s)	0.8889 (10684s)	0.8889 (12878s)

^a Random with $m = 1$ is staticMIP (static graph's policy on the dynamic graph)

1) *Estimating the Optimal Lower Bound:* We compute the Lower Bound in (6) for the AD graphs using a sample size of 100,000 graphs. We find a Lower Bound of CSR and SSR on a graph using dyMIP-C(m) and dyMIP-S(m) respectively. We set $m = 50$ and run it 200 times to cover all 100,000 graph instances in Monte Carlo Simulation. **After conducting this experiment on ADX05, the estimated the Lower Bound for SSR is 0.7240 ± 0.0055 and for CSR is 0.8889 ± 0.0019 .** These estimates are used to measure the optimality of our solutions.

2) *Discussion of Dynamic Results:* Table III summarizes our experimental results. Each experiment is conducted with 10 trials. We run our algorithm with 3 settings: pure simple attacker (dyMIP-S and $\varphi = 0$), pure competent attacker (dyMIP-C and $\varphi = 1$) and mixed-type attacker (dyMIP-M and $\varphi = 0.5$). We randomly draw a training set of 5,000 graphs for each trial. We also run the 2 proposed heuristics: voting-based heuristic (*Voting*) and clustering-based heuristic (*Kmean*). We also include a third algorithm (*Random*), which randomly picks m graphs from the samples and applies MIP(m) to that m graphs only. For each setting, we set m to 1, 10, 50, and 100 (note that Kmean does not make sense for $m = 1$ since one cluster is not meaningful). In Table III, note that *Random* with $m = 1$ is equivalent to running the MIP algorithm on one static graph (*staticMIP*), and *Voting* with $m = 1$ is applying the voting heuristic on *staticMIP*.

For *Voting*, we run it with 10 parallel tasks. For example, *Voting* with $m = 50$ with 5,000 graphs will require 100 rounds of MIP(m) if we run it sequentially, which is infeasible without parallel computing. The reported time for *Voting* is the sum of the time of 10 parallel processes. As *Random* and *Kmean* only require running MIP(m) once, we run these algorithms without parallelization. For evaluation, we test our algorithms on 100,000 randomly drawn graph samples.

Result Interpretation: We observed that static defense policy (*staticMIP* or *Random* with $m = 1$) in all 3 φ values performs poorly against all types of attackers. We are able to obtain optimal CSR ($= 0.8889$) for various settings with dyMIP-C and dyMIP-M. While for SSR ($= 0.7293$), we are able to obtain near-optimal results with dyMIP-S using *Voting* heuristic and $m = 100$. Our experiments demonstrated that *Kmean* outperforms *Random* in defending against simple attackers (yielding better SSR) under various settings. This

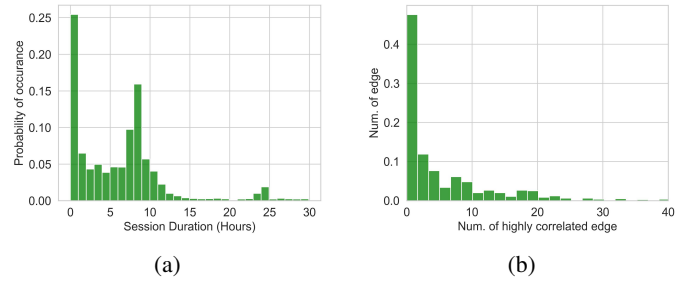


Fig. 3: (a) Number of online sessions and newly established session sampling every 1 hour (b) Histogram of a number of edges which are highly correlated (in terms of logon/off session pattern) with others.

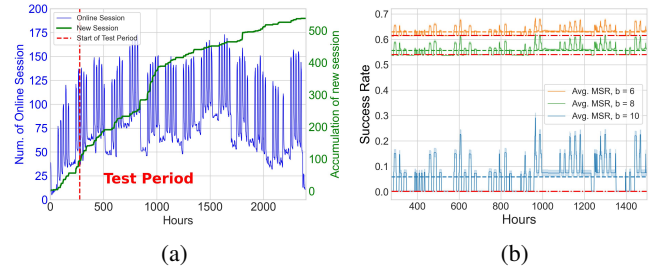


Fig. 4: (a) Number of online sessions and the accumulation of newly established session sampling every 1 hour (b) Voting with $m = 10$. The red dash-dotted lines are the optimal MSR of $b = 6, 8, 10$, listed from top to bottom.

shows that the clustering method (*Kmean*) can improve the “quality” of the samples and therefore improve the performance of MIP(m). Notably, when only considering settings that can obtain the optimal policy against a competent attacker (CSR = 0.8889), *Kmean* with $m = 100$ outperform others in term of SSR ($= 0.7781$). As the voting heuristic requires running dyMIP(m) on every possible graph in the training set to perform majority voting. Therefore, *Voting* easily outperforms the other methods in terms of SSR as it scales. However, this makes it computationally more expensive compared to *Kmean* and *Random*, which only require running dyMIP(m) once. Therefore, *Kmean* is more favourable when computational resources are limited. We remark that the success rate obtained from this experiment is quite high for the budget of 10. It is due to the high number of HasSession edges makes the graph very dense (each snapshot could contain $\approx 36,000$ HasSession edges) and hard to defend with 10 honeypots. However, in the next experiment, we will show that our approach could obtain a manageable success rate in a realistic attack graph.

D. Experiment on real dynamic AD graphs

1) *Analyzing authentication data:* We analyze in this section the temporal properties of HasSession edges in a real network and apply these properties to simulate realistic dynamic AD graphs. The dataset used in this section is sourced from an anonymous organization and referred to as **AuthOrg** data.

The dataset includes the authentication events observed over a 102-day period (from 24-06-2022 to 04-10-2022). There are 270 users, 338 computers, 14249 sessions and 1015 unique sessions in the dataset. Upon analyzing the **AuthOrg** data, we find that the random authentication scheme in Section V-C significantly diverges from realistic session patterns. Our analysis reveals that the activities of HasSession edges are not independent; rather, they exhibit correlations with each other.

From Fig. 4a, the graph exhibits a denser number of HasSession edges during working hours, indicating a higher likelihood of their appearance during this period. Secondly, HasSession edges can persist online for a period of time; for instance, some normal employee's session lasts approximately 8 hours before sign-off (Fig. 3a). Thirdly, in Fig. 3b, we present a histogram of highly correlated session profiles for each edge. For every edge e_i , we count the number of session profiles in the set $E_t \setminus e_i$ that exhibit a high correlation with $profile(e_i)$. We define two vectors as *highly correlated* if their Pearson correlation is greater than 0.5. Fig. 3b provides evidence that a significant portion of the edges exhibits high correlations with at least one other edge where the percentage of edge profiles that have at least 1 or more highly correlated edges $P[X \geq 1]$ is more than 63.8%.

In addition to correlated edges, another factor that could affect the performance of our honeypot placement algorithm is the number of new or rare link connections. New connections is connection that the establishment of unique sessions for the first time (edges that did not exist before). The accumulative new connection session line in Fig. 4a illustrates a continuous establishment of new sessions throughout the entire dataset duration. These new links can introduce alternate attack paths that were not considered during the training period, thereby contaminating the graph with unexpected paths and degrading the performance of the defense policy over time. We show in the next section that our algorithms by sampling representative behaviours across time horizons are robust to these degradation factors.

2) *Simulated AD graphs with realistic HasSession edges:* We augment the Adsimulator AD graphs in the previous sessions with the log-on and log-off data to generate more realistic AD graphs. Here, we randomly map users and computers from the authentication data to the synthetic graph which we refer to as the "AuthOrg" graph. We train our algorithm using the first 250 hours of data, excluding the initial 25 hours for training stability. The subsequent 2,073 hours are used for testing, resulting in a total of 2,448 hours of data. For ease of comprehension, the figures in this paper only display results up to 1,500 hours. We take snapshots on the graph every 0.25 hours, so we have 1000 snapshots for the training in total. In the experiments, 80% of nodes are blockable (this number could be changed and didn't affect the overall results). We used the voting heuristic with $m = 10$ for all experiments. Additionally, each set is tested on 10 trials.

3) *Result Interpretation.*: In Fig. 4b, we present the results for varying the budget, specifically $b = 6, 8, 10$. Our algorithm demonstrates strong performance with low optimality gaps

for each setting, with gaps of 0.0129, 0.0189, and 0.049 for $b = 6, 8$, and 10, respectively. We are able to archive the low attacker success rate of 5.83 % on average with a budget of 10. Notably, we observed that the attacker's success rate is influenced by two main factors: the number of online edges in the network and the presence of certain new link connections (rare links) established in the network. The experimental results exhibit a high correlation with the number of online sessions plotted in Fig. 4a. For instance, with the $b = 10$ setting, the Pearson correlation between the number of online sessions and the success rate over time is 0.5870.

Our algorithm shows overall resilience against the effects of new link connections when they are introduced linearly over time. The (Pearson) correlation between the number of new connections and the success rate over time, with $b = 10$ setting, is only 0.07. Still, we observed a burst of new connections occurring throughout week 6 (from hour 800th to 1000th, Fig. 4a), which had a noticeable impact on the overall algorithm performance, as visually observed in Fig. 4b. Our hypothesis is that during these periods there are *rare new links* that the m sample graphs do not capture.

Overall, the algorithm performs well with low optimality gaps. However, it tends to perform worse during periods with an increased number of connections in the network, such as working hours. However, it is explainable that as the graph becomes more populated with HasSession edges, the number of paths to the Domain Admin (DA) increases, subsequently elevating the attacker's success rate. Another reason is that we also assumed that we have a limited number of honeypots and having more honeypots will suppress this trend. Additionally, the presence of rare certain new link connections during the test period can also affect the algorithm's performance. This limitation can be addressed by periodically retraining the algorithm to adapt to changing network conditions or increasing the number of snapshots.

VI. CONCLUSION

This paper investigated a Stackelberg game model between an attacker and a defender on Active Directory attack graphs. The model considers two types of attackers, one of which can observe honeypots while the other cannot. The defender aims to allocate honeypots on attack graph nodes to minimize the attacker's expected success chance to reach DA. We show that the problems for both types of attackers are computationally hard and propose a mixed-integer programming (MIP) formulation to solve the problem when there is a mixed-types of attacker agents exist in the network, for both static and dynamic graphs. Our experiments showed that MIP scales well for a graph with ≈ 100 thousand nodes and ≈ 1 million edges. We show that our algorithm that combines m MIP (dyMIP(m)) instances of the graphs performs well under realistic temporal patterns. Our experimental results show that the MIP(m) algorithms produce near-optimal blocking plans, with the help of two heuristic methods based on majority voting and k-means clustering.

REFERENCES

- [1] The MITRE Corporation. BloodHound. <https://attack.mitre.org/software/S0521/>
- [2] The MITRE Corporation. Att&ck matrix for enterprise. <https://attack.mitre.org/>, 2023.
- [3] David Weston. New security features for windows 11 will help protect hybrid work. <https://www.microsoft.com/en-us/security/blog/2022/04/05/new-security-features-for-windows-11-will-help-protect-hybrid-work/>, 2022.
- [4] Andy Robbins. “bloodhound: Six degrees of domain admin. <https://github.com/BloodHoundAD/BloodHound>, 2023. Accessed: 2022-08-02.
- [5] John Dunagan, Alice X Zheng, and Daniel R Simon, ‘Heat-ray: combating identity snowball attacks using machinelearning, combinatorial optimization and attack graphs’, in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pp. 305–320, (2009).
- [6] Diksha Goel, Max Hector Ward-Graham, Aneta Neumann, Frank Neumann, Hung Nguyen, and Mingyu Guo. 2022. Defending active directory by combining neural network based dynamic program and evolutionary diversity optimisation. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1191–1199 (2022).
- [7] Diksha Goel, Aneta Neumann, Frank Neumann, Hung Nguyen, and Mingyu Guo, ‘Evolving reinforcement learning environment to minimize learner’s achievable reward: An application on hardening active directory systems’, *GECCO ’23: Genetic and Evolutionary Computation Conference*, 2023, 2023, (2023).
- [8] Mingyu Guo, Jialiang Li, Aneta Neumann, Frank Neumann, and Hung Nguyen, ‘Practical fixed-parameter algorithms for defending active directory style attack graphs’, in *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 9360–9367, (2022).
- [9] Mingyu Guo, Max Ward, Aneta Neumann, Frank Neumann, and Hung Nguyen. 2023. Scalable Edge Blocking Algorithms for Defending Active Directory Style Attack Graphs. *Proceedings of the AAAI Conference on Artificial Intelligence* (2023).
- [10] Yumeng Zhang, Max Ward, Mingyu Guo, and Hung Nguyen, ‘A scalable double oracle algorithm for hardening large active directory systems’, *The 18th ACM ASIA Conference on Computer and Communications Security (ACM ASIACCS)*, (2023).
- [11] Karel Durkota, Viliam Lis’y, Branislav Bošansk’y, and Christopher Kiekintveld, ‘Approximate solutions for attack graph games with imperfect information’, in *International Conference on Decision and Game Theory for Security*, pp. 228–249. Springer, (2015).
- [12] Karel Durkota, Viliam Lis’y, Branislav Bošansk’y, and Christopher Kiekintveld, ‘Optimal network security hardening using attack graph games’, in *Twenty-Fourth International Joint Conference on Artificial Intelligence*, (2015).
- [13] Karel Durkota, Viliam Lis’y, Branislav Bošansk’y, Christopher Kiekintveld, and Michal Pěchouček, ‘Hardening networks against strategic attackers using attack graph games’, *Computers and Security*, 87, 101578, (2019).
- [14] Stephanie Milani, Weiran Shen, Kevin S Chan, Sridhar Venkatesan, Nandi O Leslie, Charles Kamhoua, and Fei Fang, ‘Harnessing the power of deception in attack graph-based security games’, in *International Conference on Decision and Game Theory for Security*, pp.147–167. Springer, (2020).
- [15] Ondrej Lukas and Sebastian Garcia, ‘Deep generative models to extend active directory graphs with honeypot users’, *arXiv preprint arXiv:2109.06180*, (2021).
- [16] Aditya Shinde, Prashant Doshi, and Omid Setayeshfar, ‘Cyber attack intent recognition and active deception using factored interactive pomdps’, in *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, pp. 1200–1208, (2021).
- [17] Per-Erik Danielsson, ‘Euclidean distance mapping’, *Computer Graphics and image processing*, 14(3), 227–248, (1980).
- [18] Wassily Hoeffding, ‘Probability inequalities for sums of bounded random variables’, *The collected works of Wassily Hoeffding*, 409–426, (1994).